

Tugas Praktikum

Message Passing Interface (MPI)

PENDAHULUAN

Modul ini berisi lima tugas praktikum MPI untuk memahami komunikasi dasar antar proses: Scatter, Gather, kombinasi Scatter+Gather, serta Reduce vs Allreduce. Praktikum disusun bertahap dari distribusi data, transformasi lokal, hingga agregasi global. Setiap tugas menyertakan petunjuk eksekusi, starter code, ruang tabel hasil/grafik, dan pertanyaan diskusi & analisis.

Tugas Praktikum 1 — Scatter Sederhana

Tujuan

- Memahami distribusi data dari rank 0 ke seluruh proses menggunakan Scatter.
- Mengamati perilaku urutan output pada eksekusi paralel.

Spesifikasi

1. Proses 0 memiliki list [10, 20, 30, 40].
2. Data dibagi rata ke 4 proses dengan scatter.
3. Setiap proses mencetak data yang diterimanya.

Starter Code

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Pastikan jumlah proses = 4
if size != 4:
    if rank == 0:
        print("Harap jalankan dengan -n 4")
        raise SystemExit

data = [10, 20, 30, 40] if rank == 0 else None
recv = comm.scatter(data, root=0)

print(f"Rank {rank} menerima {recv}")
```

Tangkapan Layar

Sisipkan screenshot hasil terminal eksekusi (semua proses).

Tabel Hasil

Rank	Data Diterima
0	
1	
2	
3	

Pertanyaan Diskusi

- Apa output yang dihasilkan tiap proses? (Harap tuliskan hasil observasi)
- Mengapa urutannya bisa berbeda saat dicetak? (Bahas sifat output paralel & buffering stdout)

Tugas Praktikum 2 — Gather Sederhana

Tujuan

- Memahami pengumpulan data dari semua proses ke root menggunakan **Gather**.
- Mengamati perubahan hasil saat jumlah proses berubah.

Spesifikasi

1. Tiap proses mengirim data rank * 5.
2. Data dikumpulkan di proses 0 dan dicetak sebagai list.

Starter Code

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

send_val = rank * 5
gathered = comm.gather(send_val, root=0)

if rank == 0:
    print(f"Hasil gather di rank 0: {gathered}")
```

Tangkapan Layar

Sisipkan hasil untuk -n 4 dan -n 6.

Tabel Hasil

Proses	Hasil List di Rank 0
4	
6	

Pertanyaan Diskusi

- Apa isi list hasil di proses 0 jika ada 4 proses?
- Bagaimana hasilnya jika jumlah proses = 6?
- Jelaskan mengapa urutan elemen sesuai rank.

Tugas Praktikum 3 — Scatter + Gather (Kombinasi)

Tujuan

- Menggabungkan distribusi data (scatter), komputasi lokal, dan pengumpulan hasil (gather).
- Memvalidasi transformasi elemen (kuadrat) per-proses.

Spesifikasi

1. Proses 0 memiliki list [1, 2, 3, 4].
2. Data di-scatter ke 4 proses.
3. Setiap proses menghitung kuadrat dari datanya.
4. Hasil di-gather kembali ke proses 0.

Starter Code

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if size != 4:
    if rank == 0:
        print("Harap jalankan dengan -n 4")
        raise SystemExit

data = [1, 2, 3, 4] if rank == 0 else None # Uji juga: [2, 4, 6, 8]
x = comm.scatter(data, root=0)
y = x * x
hasil = comm.gather(y, root=0)

if rank == 0:
    print(f"Input : {data}")
    print(f"Output : {hasil}")
```

Tangkapan Layar

Lampirkan hasil untuk input [1,2,3,4] dan [2,4,6,8].

Tabel Hasil

Rank	Input	Output (Kuadrat)

Pertanyaan Diskusi

1. Apa hasil akhir yang dikumpulkan di proses 0? untuk [1,2,3,4].
2. Bagaimana jika list awal diganti [2,4,6,8]?
3. Jelaskan korespondensi indeks (rank ↔ posisi data).

Tugas Praktikum 4 — Hitung Jumlah dengan Scatter + Gather (Bonus)

Tujuan

- Menerapkan transformasi lokal dan agregasi total di rank 0.
- Melatih validasi hasil komputasi terdistribusi.

Spesifikasi

1. Proses 0 punya list [5, 10, 15, 20] (4 elemen).
2. Data di-scatter.
3. Tiap proses menghitung $2 \times$ data.
4. Hasil di-gather, lalu dijumlahkan di proses 0.

Starter Code

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

if size != 4:
    if rank == 0:
        print("Harap jalankan dengan -n 4")
        raise SystemExit

data = [5, 10, 15, 20] if rank == 0 else None
x = comm.scatter(data, root=0)
local = 2 * x
kumpul = comm.gather(local, root=0)

if rank == 0:
    total = sum(kumpul)
    print(f>Data awal      : {data}")
    print(f"x2 per-proses: {kumpul}")
    print(f>Total akhir   : {total}") # Ekspektasi: 100
```

Tangkapan Layar

Sisipkan hasil yang memperlihatkan total akhir = 100.

Tabel Hasil

Rank	Input	Output (x2)

Jawaban Singkat (Validasi)

Total = 100 karena $(2 \times 5) + (2 \times 10) + (2 \times 15) + (2 \times 20) = 10 + 20 + 30 + 40$.

Tugas Praktikum 5 — Reduce vs Allreduce (MAX & Rata-Rata Global)

Tujuan

- Membedakan Reduce (hasil hanya di root) vs Allreduce (hasil diketahui semua proses).
- Menggunakan MPI.MAX untuk nilai terbesar & MPI.SUM untuk total/rata-rata global.
- Mengagregasi data acak skala besar tanpa mengirim seluruh array.

Spesifikasi

1. **Bagian A (MAX):** Ganti MPI.SUM → MPI.MAX. Tentukan proses mana yang punya nilai terbesar.
2. **Bagian B (AVG):** Hitung rata-rata global = (global sum) / (size) menggunakan Allreduce.
3. **Bagian C (Total 1000 bilangan acak/proses):** Gunakan Allreduce untuk menghitung total global.

Starter Code

```
from mpi4py import MPI
import numpy as np

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# ----- A: MAX dengan Reduce -----
my_val = rank * 7 + 3
max_val = comm.reduce(my_val, op=MPI.MAX, root=0)
if rank == 0:
    print(f"[REDUCE-MAX] max = {max_val}")
    print("Proses dengan nilai terbesar biasanya rank terakhir (fungsi naik vs rank).")

# ----- B: Rata-rata global via Allreduce -----
val_for_avg = rank + 1
global_sum = comm.allreduce(val_for_avg, op=MPI.SUM)
global_avg = global_sum / size
print(f"[ALLREDUCE-AVG] Rank {rank}: sum={global_sum}, avg={global_avg}")

# ----- C: Total 1000 bilangan acak/proses via Allreduce -----
rng = np.random.default_rng(seed=12345 + rank)
nums = rng.random(1000)
local_sum = float(np.sum(nums))
total_sum = comm.allreduce(local_sum, op=MPI.SUM)

if rank == 0:
    print(f"[ALLREDUCE-SUM] Total 1000*size bilangan acak = {total_sum:.6f}")
```

Tangkapan Layar

Sisipkan hasil yang menunjukkan nilai MAX, SUM, dan AVG

Tabel Hasil

Bagian	Metrik	Nilai
A	MAX (Reduce)	
B	SUM (Allreduce)	
B	AVG (SUM/Size)	
C	TOTAL (Allreduce)	

Pertanyaan Diskusi

1. Jelaskan perbedaan konseptual & praktis Reduce vs Allreduce.
2. Mengapa MAX (fungsi naik terhadap rank) memunculkan nilai terbesar di rank terbesar?
3. Mengapa Allreduce efisien untuk agregasi besar (mis. 1000 bilangan acak/proses) dibanding mengirim array mentah?

PENUTUP

Dengan menyelesaikan kelima praktikum ini, mahasiswa diharapkan:

1. Memahami dasar komunikasi MPI: Scatter, Gather, Reduce, Allreduce.
2. Mampu merancang alur distribusi → komputasi lokal → agregasi hasil.
3. Menyadari implikasi non-determinisme urutan output pada program paralel.
4. Siap mengembangkan studi kasus yang lebih kompleks (prefix-sum paralel, histogram global terdistribusi, dsb).